

YALU CAI

✉ yc1795@cs.rutgers.edu | [yc2454.github.io](https://github.com/yc2454) | [yc2454](https://www.linkedin.com/in/yc2454) | [yalu-cai](https://www.linkedin.com/in/yalu-cai)

Research interests: automated reasoning and formal verification for systems software: SMT-based proof generation, proof-carrying code, abstract interpretation, and verified checkers and compilers in Lean 4.

Education

Rutgers University · PhD in Computer Science, advised by Prof. Santosh Nagarakatte	Sept. 2025 – Present
Duke University · MS in Computer Science, GPA 3.84/4.0	Aug. 2020 – May 2022
Cornell University · BS in Biological Engineering, GPA 3.90/4.3	Aug. 2018 – May 2020
Zhejiang University · BE in Biological Engineering, GPA 3.77/4.0	Sept. 2016 – Jul. 2018

Publications

Proof-Carrying-BPF Programs for the eBPF Verifier

Yalu Cai, Santosh Nagarakatte

eBPF '26: The 4th Workshop on eBPF and Kernel Extensions, Prague, Czech Republic, Sept. 2026

Research Experience

Proof-Carrying-BPF Programs for the eBPF Verifier | *Rutgers, eBPF '26*

Sept. 2025 – Present

- Proposed proof-carrying BPF (PCB): the compiler backend ships bytecode with SMT proofs for every point where the kernel verifier's abstract domains would reject, accepting more safe programs without a larger trusted base.
- The kernel matches each refinement condition it rebuilds to a bundled proof by hash and validates it with a small in-kernel checker, all in a single bpf syscall.
- Unlike SOSP '25's BCF, needs no verifier suspend/resume or kernel-userspace round trips: loads all 360 previously rejected production objects (Cilium, Calico, BCC, Inspektor Gadget), $3.4\times$ faster on average.

Verified Proof Checker for BCF Certificates | *Lean 4*

Jun. 2026 – Present

- Built an independent checker for BCF's eBPF proof certificates (~ 50 bit-vector and boolean rules from cvc5's CPC calculus, 151 rewrites, bit-blasting).
- Proved in Lean that every accepted proof certifies an unsatisfiable goal, which surfaced soundness defects in its rewrite table.
- Defined a subset of C semantics in Lean for C emission. Aiming to prove every proof accepted by the verified checker is accepted by the emitted C.

koit: A Verifier-Aware Source Language for eBPF | *Lean 4*

Jun. 2026 – Present

- Designing a language whose types state the kernel verifier's conditions (refinement types, typed memory regions, resource management), so well-typed programs compile to bytecode it accepts; compiler correctness proofs in Lean are in progress.

Verified Translator from EVM Bytecode to eWASM | *Coq, Duke (MS)*

Sept. 2021 – May 2022

- Built a translator from a subset of EVM bytecode to WebAssembly in Coq aiming for proving semantic preservation.

Projects

Alivio | *Rust, cvc5, SMT-LIB*

Sept. 2025 – Present

- Open-source Rust mirror of the Linux eBPF verifier that generates PCB's proof bundles; matches the kernel's verdict on all 3,267 BPF selftests, reproducing its exact state pruning, liveness, and precision analyses.
- Prototyped a spec language for the verifier's abstract operators that compiles to both Rust and SMT-LIB.

Industry Experience

Software Engineer

Jun. 2022 – Aug. 2025

CertiK LLC

New York, NY, USA

- Designed and built compilers from Solidity to Dafny and Boogie; helped design Swivel, their shared IR.
- Extended Solidity with a JML-style specification language; deductively verified Solidity projects in Dafny.
- Wrote Python tooling to build and run the verification toolchain on Solidity projects, greatly shortening test cycles.

Technical Skills

Formal methods: Lean 4, Coq/Rocq, Dafny, Boogie, SMT solvers (cvc5, Z3), SMT-LIB, abstract interpretation

Languages: Rust, Python, C/C++, OCaml, Haskell

Systems & tools: Linux kernel eBPF, LLVM/Clang, QEMU/KVM, Git, Docker, AI coding agents (Claude Code)